Hoja de Ejercicios 3. Procesos y Planificación

Ejercicios Prácticos

Los ejercicios marcados con el icono 💻 son prácticos y deben realizarse en el laboratorio. Los ejercicios prácticos de esta hoja requieren el entorno de usuario básico (shell) y de desarrollo (compilador, editores y depurador). Además, algunos ejercicios necesitan acceso de superusuario que está disponible en las máquinas virtuales de la asignatura.

Control de Procesos

💻 ***Ejercicio 1.*** ps(1) permite ver los procesos del sistema y su estado. Estudiar la página de manual y determinar las opciones necesarias para:

- Mostrar todos los procesos del usuario actual en formato extendido. (Nota: usar la variable de entorno USER).
- Mostrar los procesos del sistema, incluyendo el identificador del proceso, el identificador del grupo de procesos, el identificador de sesión, el estado y el comando con todos sus argumentos.
- Observar el identificador de proceso, grupo de procesos y sesión de los procesos. ¿Qué identificadores comparten la *shell* y los programas que se ejecutan en ella (usar la opción -H para identificar fácilmente la relación entre procesos)?
- ¿Cuál es el identificador de grupo de procesos cuando se crea un nuevo proceso? Usar el comando sleep para poder verlo fácilmente en la salida del comando ps.

💻 ***Ejercicio 2.*** Antes de ejecutar un programa hay que generarlo a partir del código fuente. Usando un programa en C sencillo (por ejemplo, uno que imprima un mensaje por pantalla):

- Compílalo con `gcc --save-temps -o programa programa.c` y localiza los ficheros intermedios que genera (preprocesado, ensamblador y objeto) además del ejecutable final. ¿Qué contiene cada uno?
- Usa `ldd programa` para consultar con qué bibliotecas dinámicas se ha enlazado el ejecutable.
- Usa `nm programa` y `objdump -d programa` para localizar, respectivamente, la tabla de símbolos y el código ensamblador correspondiente a la función `main`.

💻 ***Ejercicio 3.*** El entorno de un proceso es una tabla de pares (nombre, valor) que se hereda del proceso padre en su creación.

- Desde la shell, define una variable con `MIVAR=valor` (sin `export`) y comprueba con `echo $MIVAR` que la shell la conoce. Escribe un pequeño programa en C que llame a `getenv("MIVAR")` e imprima el resultado. ¿La ve el programa? ¿Por qué?
- Repite la prueba exportando la variable (`export MIVAR=valor`) antes de ejecutar el programa. ¿Cambia el resultado?
- Amplía el programa para que, usando `putenv()`, modifique el valor de `MIVAR` y después lo elimine con `unsetenv()`, comprobando en cada paso el valor con `getenv()`. Al terminar el programa, comprueba con `echo $MIVAR` en la shell el valor de la variable. ¿Se ha visto afectada por los cambios hechos en el proceso hijo?

💻 ***Ejercicio 4.*** Escribir un programa que muestre los identificadores del proceso (PID, PPID, PGID y SID), el identificador del usuario y grupo, y su directorio de trabajo actual.

💻 ***Ejercicio 5.*** Todo proceso tiene un identificador de usuario real (el del usuario que lo lanzó) y uno efectivo (el usado por el SO para el control de acceso), que normalmente coinciden.

- Amplía el programa del Ejercicio 4 para que además imprima el UID y GID *efectivos* con `geteuid()` y `getegid()`.

- Compílalo, copia el ejecutable, hazlo propiedad de root y activa su bit *setuid* (`chown root:root fichero; chmod u+s fichero`).
- Ejecuta primero la copia del programa *sin* el bit setuid y después la copia *con* el bit setuid, en ambos casos como un usuario distinto de root. Compara el UID real y el efectivo en cada caso. ¿Cuándo difieren? Relaciona el resultado con el propósito de programas como `passwd`, que necesitan privilegios de root solo para una parte de su ejecución.

Ejercicio 6. Considera la siguiente orden de la línea de comandos:

```
echo "12345" > /proc/$$/fd/1
```

Discuta razonadamente cuál es el resultado esperado de la ejecución.

💻 ***Ejercicio 7.*** Consulta el tipo de fichero y contenido de los siguientes ficheros del proceso de la shell actual (/proc/$$) completa la siguiente tabla:

Fichero	**Tipo** (enlace, dir,…)	**Descripción** (contenido/propósito)
cmdline		
cwd		
environ		
exe		
fd		
limits		
maps		
root		

Creación y Gestión de Procesos

💻 ***Ejercicio 8.*** Escribir un programa que cree un proceso hijo con las siguientes características:

- El programa recibirá dos argumentos en la forma:

  ```
  ./ejercicio8 <segundos_padre> <segundos_hijo>
  ```

- El hijo creará su propia sesión, imprimirá sus identificadores, esperará <segundos hijo> (segundo argumento) con la llamada sleep(3); y terminará.
- El padre imprimirá sus identificadores, esperará <segundos padre> (primer argumento) con la llamada sleep; y terminará.

La llamada usada por el hijo para crear su propia sesión es `pid_t setsid(void)`. Un proceso que llama a `setsid()` deja de pertenecer a la sesión y al grupo de procesos que heredó de su padre y pasa a ser el único miembro (y líder) de una sesión y un grupo de procesos nuevos, con SID y PGID iguales a su propio PID. Como consecuencia dejará de recibir las señales que el terminal envía al grupo de procesos en primer plano (p. ej. la que genera Ctrl-C) y, si su sesión original termina, no será adoptado como huérfano dentro de ella. Solo puede invocarse con éxito si el proceso no es ya el líder de un grupo de procesos.

Ejemplo de salida:

```
$ ./eje8 2 1
[Padre] PID=1616, PPID=1362, PGID=1616, SID=1362. Durmiendo 2s
[Hijo] PID=1617, PPID=1616, PGID=1617, SID=1617. Durmiendo 1s
[Hijo] Terminado.
[Padre] Terminado.
```

Considera las siguientes ejecuciones, observa los procesos con la orden ps -lHu $USER y completa la siguiente tabla para los procesos relacionados con el ejercicio:

Orden	**Proceso**	**PID**	**PPID, CMD (padre)**	**Estado**
./eje8 600 1&				
./eje8 1 600&				

Orden	Proceso	PID	PPID, CMD (padre)	Estado

NOTA: no todas las filas son necesarias

Explica razonadamente el estado de los procesos en el primer caso (el hijo termina antes) y el PPID de los procesos en el segundo caso (el padre termina antes), identifica el proceso padre con la ayuda del comando ps(1).

Ejecuta el programa con un tiempo de espera largo (ej. ./eje8 60 60), antes de que terminen las llamadas a sleep(3) presiona Ctrl-C en el terminal. ¿Qué ocurre? ¿Mueren ambos procesos? ¿Por qué?

💻***Ejercicio 9.*** kill(1) permite enviar señales a un proceso o grupo de procesos por su identificador (pkill(1) permite hacerlo por nombre de proceso). Estudiar la página de manual y las señales que se pueden enviar a un proceso.

💻***Ejercicio 10.*** En un terminal, arrancar un proceso de larga duración (ej. sleep 600). En otra terminal, enviar diferentes señales al proceso (terminar, interrumpir, parar, continuar) y comprobar el comportamiento. Las señales involucradas son, respectivamente, SIGTERM, SIGINT, SIGSTOP y SIGCONT; puedes enviarlas por nombre con `kill -SIGTERM <PID>` o de forma equivalente con `kill -TERM <PID>`. Si el proceso no responde a SIGTERM se puede recurrir a SIGKILL, que no puede ser ignorada ni capturada.

Ejercicio 11. Determina las opciones adecuadas para el comando kill(1) para terminar (SIGINT) los dos procesos que se crean en la ejecución del programa del ejercicio 8. Recuerda que, al llamar a `setsid()`, el hijo abandona el grupo de procesos y la sesión del padre y crea los suyos propios: por tanto, una única orden `kill` dirigida al grupo de procesos del padre (`kill -INT -<PGID_padre>`) **no** alcanzará al hijo, y es necesario indicar explícitamente el PID (o el nuevo PGID) de cada uno de los dos procesos.

💻***Ejercicio 12.*** Por defecto, cuando un proceso recibe SIGINT (Ctrl-C) termina. Sin embargo, un proceso puede instalar su propio manejador con `sigaction()` para reaccionar de otra forma.

- Escribe un programa que instale un manejador para SIGINT que, en lugar de terminar el proceso, imprima un mensaje y cuente cuántas veces ha sido invocado; el proceso debe terminar (`_exit()`) solo tras haber recibido la señal 3 veces. El resto del tiempo el proceso debe permanecer bloqueado esperando señales, por ejemplo con `pause()` dentro de un bucle.
- Compílalo, ejecútalo en primer plano y comprueba con Ctrl-C (o, desde otro terminal, con `kill -INT <PID>`) que hacen falta exactamente tres señales para que termine.
- ¿Qué ocurre si en su lugar envías SIGKILL al proceso? ¿Por qué SIGKILL no puede interceptarse de la misma forma?

💻***Ejercicio 13.*** Escribir un programa que ejecute otro programa (ejecutable y argumentos) que se pasará como argumento. El programa creará un proceso hijo que ejecutará el programa dado en el argumento con la función execvp(3). El proceso padre esperará que termine el hijo e imprimirá su código de salida. Ejemplos de ejecución:

```
$ ./eje13 cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
...
El proceso hijo terminó con código de salida 0

$ ./eje13 cat /etc/shadow
cat: /etc/shadow: Permission denied
El proceso hijo terminó con código de salida 1

$ (./eje13 sleep 3600&) ; sleep 1 ; pkill -SIGKILL sleep
El proceso hijo terminó por señal 9
```

Nota: Considerar cómo deben pasarse los argumentos en cada caso para que sea sencilla la implementación. Por ejemplo: ¿qué diferencia hay entre: ./eje13 ps -el y ./eje13 “ps -el”?

💻***Ejercicio 14.*** wait() y waitpid() sin opciones bloquean al padre hasta que el hijo termina. La llamada `waitpid(pid, &status, WNOHANG)` en cambio no bloquea: si el hijo indicado todavía no ha terminado devuelve 0 inmediatamente, permitiendo al padre seguir haciendo otro trabajo mientras comprueba

periódicamente si el hijo ya ha terminado (*polling*).

- Escribe un programa en el que el padre cree un hijo que se limite a dormir unos segundos con sleep(3) y terminar.
- El padre, en lugar de bloquearse con wait(), debe comprobar en un bucle con `waitpid(pid, &status, WNOHANG)` si el hijo ha terminado; mientras no lo haya hecho, debe imprimir un mensaje indicando que sigue esperando y dormir un segundo antes de volver a comprobarlo.
- En cuanto `waitpid()` devuelva el PID del hijo, el padre debe imprimir cuántas comprobaciones ha hecho y el código de salida del hijo.

Ejercicio 15. Dibuja el esquema jerárquico de los procesos creados en la ejecución del siguiente programa con argc=3:

```
void main(int argc, char *argv[])
{
    int i;
    for(i=1; i<=argc; i++)
    {
        pid_t pid = fork();
        ...
    }
    return 0;
}
```

💻***Ejercicio 16.*** Escribir un programa que genere un fichero con 5 procesos concurrentes con la siguiente estructura:
El proceso padre creará el fichero de salida output.txt y escribirá el primer segmento de 5 ceros (00000). A continuación, creará 5 hijos y esperará a que termine cada uno mostrando el PID y número de hijo. Cada hijo realiza las siguientes acciones:

- Abrirá el archivo y desplazará el puntero de escritura el offset correspondiente.
- Escribirá la secuencia correspondiente (“11111”, “22222”…). Esta parte puede implementarse de dos formas:
  - Creando una cadena con la secuencia correspondiente, con sprintf(3); y escribiendo la cadena en el archivo.
  - Definiendo un array con cada posición inicializada al carácter correspondiente al hijo (char c = ‘0’ + index) ; y escribiendo el array en el archivo

El fichero queda dividido en 6 bloques de 5 caracteres cada uno (uno del padre y uno por cada hijo). El hijo *i* (numerado de 1 a 5) debe escribir su bloque en el offset `5 * i`; por ejemplo, el hijo 2 escribe “22222” en el offset 10 (= 5 × 2):

- Padre → offset 0 → “00000”
- Hijo 1 → offset 5 → “11111”
- Hijo 2 → offset 10 → “22222”
- Hijo 3 → offset 15 → “33333”
- Hijo 4 → offset 20 → “44444”
- Hijo 5 → offset 25 → “55555”

Ejemplo de ejecución:

```
$ ./eje10
El proceso hijo 5 con PID 1890 terminó correctamente
El proceso hijo 4 con PID 1889 terminó correctamente
El proceso hijo 3 con PID 1888 terminó correctamente
El proceso hijo 2 con PID 1887 terminó correctamente
El proceso hijo 1 con PID 1886 terminó correctamente
Escritura completada en output.txt

$ cat output.txt
00000111112222233333444445555
```

Pista: Usar el código de retorno en cada hijo igual al número de tarea (exit(2)) de forma que con la llamada wait(2) se puedan obtener ambos valores: PID e ID del hijo.

Ejercicio 17. Considera el programa descrito en el Ejercicio 16. Dado que la tabla de descriptores se hereda ¿Es posible usar el descriptor de fichero abierto por el padre en los hijos?
Ejercicio 18. Considere el código que se muestra a continuación:

```
int global;

void main() {
    int local=3;
    pid_t pid;

    global=10;
    pid = fork();

    if (pid == 0 ) {
        global = global + 5;
        local = local + 5;
    }
    else {
        wait(NULL);
        global += 10;
        local += 10;
    }

    printf("global:%d local:%d\n", global, local);
}
```

Determine qué mensajes se imprimirán en la terminal. ¿Es posible que el resultado de la ejecución del programa cambie según el orden en que se ejecuten los procesos padre e hijo?. **Nota:** suponer que todas las llamadas al sistema se ejecutan exitosamente.
Ejercicio 19. Considere el código que se muestra a continuación:

```
int a = 3;

void main() {
    int b=2;

    for (i=0;i<4;i++) {
        pid_t p=fork();

        if (p==0) {
            b++;
            execlp("/usr/bin/sleep", "/usr/bin/sleep", "2", NULL);
            a++;
        }
        else {
            wait(NULL);
            a++;
            b--;
        }
    }

    printf("variables - a:%d b:%d\n", a, b);
}
```

Determine qué mensajes se imprimirán en la terminal. ¿Cuántos procesos se crean en total? ¿Cuántos coexisten en el sistema como máximo?. **Nota:** suponer que todas las llamadas al sistema se ejecutan exitosamente.

Planificación de Procesos

💻***Ejercicio 20.*** La política de planificación y la prioridad de un proceso puede consultarse y modificarse con chrt(1). Adicionalmente, nice(1) y renice(1) permiten ajustar el valor de *nice* de un

proceso. Consultar la página de manual de ambos comandos. Ten en cuenta que estas órdenes pueden requerir ejecutarse con sudo, ya que fijar una política de tiempo real o disminuir el valor de nice (aumentar la prioridad) son operaciones privilegiadas:

- Ejecutar una shell (/usr/bin/bash) con nice +10.
- Ejecutar una shell (/usr/bin/bash) con nice -10.
- Ejecutar una shell (/usr/bin/bash) con política de planificación a SCHED_FIFO y prioridad 12.
- Ejecutar una shell (/usr/bin/bash) con política de planificación a SCHED_RR y prioridad 12. Repite la comprobación con `chrt -p` y compárala con el caso SCHED_FIFO: ¿en qué se diferencian estas dos políticas de tiempo real cuando hay varios procesos con la misma prioridad?

En todos los casos, comprobar la salida del comando chrt -p <PID de la nueva shell> y ps -al y completar la siguiente tabla.

Proceso	Política de planificación	Prioridad	nice	Requiere privilegios de root
Shell con nice +10				
Shell con nice -10				
Shell fifo y prio 12				

- Considerando que los tres primeros procesos están listos para ejecutarse determina el orden en el que se ejecutarán.
- ¿Cuál es la prioridad y nice de los procesos que se crean en la nueva shell? (p.ej. ejecuta el comando sleep 600 y determina sus prioridad, política de planificación y nice)

Ejercicio 21. En un sistema con los siguientes procesos (tiempos en segundos):

Proceso	Llegada	CPU	E/S	CPU	E/S
P1	0	1	5	1	-
P2	1	3	1	1	1
P3	0	5	4	1	-
P4	3	3	2	1	1

Determina los tiempos de retorno (*turnaround*) y de espera para cada proceso y la productividad (*throughput)* del sistema para las siguientes políticas de planificación:

- *First Come First Serve* (FCFS)
- *Shortest Job First* (SJF)
- *Round Robin* (RR) con cuanto de 3s
- *Round Robin* (RR) con cuanto de 1s

Ejercicio 22. Considere un sistema monoprocesador con una política de planificación de procesos de 3 niveles con realimentación. Los procesos que agotan el cuanto bajan de nivel y los que ceden la CPU antes de agotarlo son promocionados. Además, cuando un proceso acaba una operación de E/S se añade a la cola de espera de mayor prioridad. Cada nivel usa una política round robin cuyos cuantos de tiempo son 2, 4 y 8, respectivamente para cada nivel.

Al principio hay 3 procesos en la cola del nivel 1 (máxima prioridad) y el resto de colas están vacías. Los procesos tienen el siguiente patrón de ejecución que se repite indefinidamente:

Proceso	CPU	E/S
P1	3	5
P2	8	5
P3	5	5

Dibuja un diagrama de gantt para la ejecución para los primeros 30s que muestre además el estado de las colas de espera de cada nivel. Calcula los tiempos de espera de cada proceso.

Ejercicio 23. Repetir el problema anterior con las siguientes configuraciones:

- MLF de 10 niveles con el cuanto de cada nivel (Q_i) $Q_i = 2 \cdot i$
- MLF de 10 niveles con el cuanto de cada nivel (Q_i) $Q_i = 2$ (constante en todos los niveles).

Ejercicio 24. El planificador EEVDF de Linux asigna a cada proceso un tiempo virtual de elegibilidad *ve* y un plazo virtual *vd*. Cuando un proceso entra en el sistema (o completa su ráfaga T_S), se fija ve = V(T) y vd = ve + T_S/w, donde w es su peso y V(T) es el tiempo virtual global, que avanza según V(T) +=

(tiempo real consumido)/(suma de pesos de los procesos activos). En cada instante de selección, EEVDF elige entre los procesos *elegibles* (aquellos con ve ≤ V(T)) el que tenga menor vd.

Considera 3 procesos que llegan simultáneamente en el instante 0 y permanecen siempre activos (no se bloquean), con los siguientes pesos y duraciones de ráfaga T_S (en ticks):

Proceso	**Peso (w)**	**T_S (ticks)**
A	4	2
B	2	2
C	2	1

Todos entran con ve = 0 en el instante V(T) = 0. En caso de empate en vd, se elige el proceso cuyo nombre sea alfabéticamente menor.

- Calcula ve y vd iniciales de los tres procesos y determina qué proceso se ejecuta primero.
- Continúa la traza turno a turno (recalculando V(T), ve y vd tras cada ráfaga completada) hasta que se hayan consumido 10 ticks de tiempo real en total. Indica el orden de ejecución resultante.
- Suma los ticks de CPU recibidos por cada proceso en esos 10 ticks. ¿Es exactamente proporcional a su peso (recuerda que la cuota "justa" de A, B y C sería 10·4/8, 10·2/8 y 10·2/8 ticks respectivamente)? ¿Por qué EEVDF solo garantiza un reparto proporcional *a largo plazo* y no en cada instante?